

- 입력 문장의 단어를 좌우 양방향으로 처리해 문맥 정보를 모델링 → 인코더 계층만을 이용해 학습

7.3.1 사전 학습 방법



- BERT는 대규모 데이터로 사전학습된 모델 → 전이학습에 주로 이용됨.
- 일부/전체를 다른 작업에서 재사용해 적은 양의 데이터로 높은 정확도 달성 가능

✓ MLM(Masked Language Modeling, 마스킹된 언어 모델링)

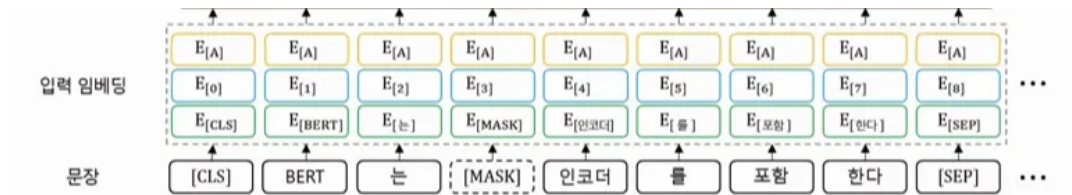
- 입력 문장에서 임의로 일부 단어를 마스킹하고 해당 단어 예측
⇒ 누락된 단어를 추론하는 능력 ↑
- 이 과정에서 양방향 문맥 정보 참고

✓ NSP(Next Sentence Prediction, 다음 문장 예측)

- 2번째 문장이 1번째 문장의 다음에 오는 문장인지 여부 판단
- 두 문장이 연속적 관계? 아니야? 판단
⇒ 두 문장 간의 관계 학습 + 문장 간의 의미적인 유사성 파악

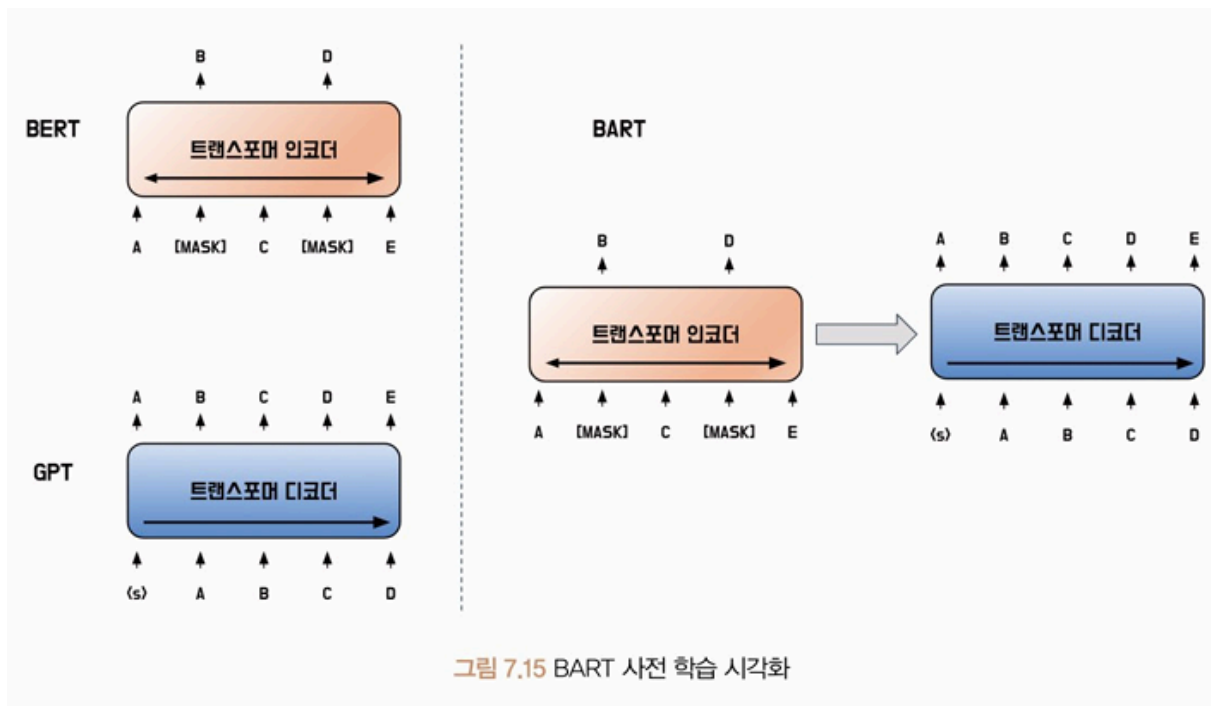
✓ 토큰 추가

- 입력 문장에 특수한 토큰을 추가해 모델 학습&추론 과정에서 필요한 정보 제공
 - CLS 토큰 - 입력 문장 시작 부분에 추가,
문장 분류 작업에 사용 / 어떤 유형의 문장인지 미리 파악 가능
 - SEP 토큰 - 입력 문장 내에서 두개 이상 문장 구분
 - MASK 토큰 - 입력 문장 내 임의로 선택된 단어를 가리키는 특별한 토큰, 마스킹에 사용



7.4 BART

(Bidirectional Auto-Regressive Transformer)



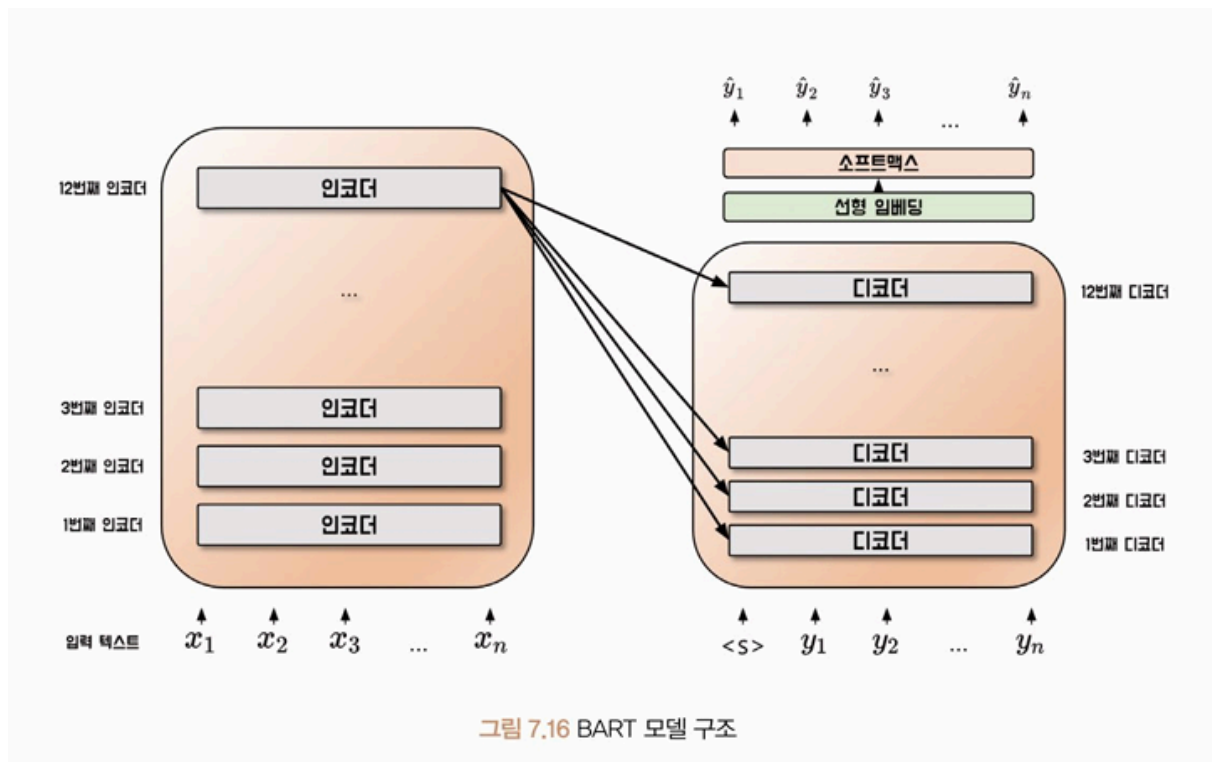
- 인코더, 디코더 모두 사용
 - ⇒ 문장 생성, 번역에 적합 / 문장 독해, 문서 요약, 질의응답 등에 있어 높은 성능을 보임.

✅ Seq2Seq (Sequence-to-Sequence, 시퀀스-시퀀스)

- BERT 인코더 + GPT 디코더
- BERT 인코더 : 입력 문장에서 일부 단어를 무작위로 마스킹, 마스킹된 단어 맞추게 학습
⇒ 입력 : 노이즈 추가된 텍스트 데이터
- GPT 디코더 : 입력 - 문장의 이전 토큰, 다음에 올 토큰 맞추게 학습
⇒ 입력 : 원본 텍스트, 학습 목표 : 원본 텍스트 생성

✅ 노이즈 제거 오토 인코더(Denoising Autoencoder) 사전 학습

- 입력 데이터에 노이즈 추가
- 원본 데이터를 복원하도록 학습
- BERT의 인코더와 GPT의 디코더가 학습하는 방법을 일반화해 학습



✅ 트랜스포머 vs. BART

- 트랜스포머 : 인코더의 모든 계층과 디코더의 모든 계층 사이의 어텐션 연산 수행
- BART : 인코더의 모든 계층과 디코더의 각 계층 사이만 어텐션 연산 수행 ; 정보 전달 최적화, 메모리 ↓
 - 인코더 : 입력 문장의 각 단어 임베딩

- 마지막 계층 : 입력 문장 전체의 의미를 가장 잘 반영하는 벡터 생성 ⇒ 디코더 출력 문장 생성 시 참고
- 디코더의 각 계층 :  벡터 + 이전 계층에서 생성된 출력 문장 정보 ⇒ 출력 문장 생성

7.4.1 사전 학습 방법



✅ BART 사전 학습에 사용되는 여러 노이즈 기법

1. 토큰 마스크 (Token Masking)
 - BERT에서 사용한 MLM과 동일한 기법
 - 입력 문장의 일부 토큰을 마스크 토큰으로 치환
 - 문장 내에서 어떤 단어가 중요한 역할을 하는지 학습 가능
⇒ 문맥을 이해하고 문장 내에서 중요한 정보를 추출하는 능력 향상
2. 토큰 삭제 (Token Deletion)
 - 입력 문장의 일부 토큰을 삭제
 - 토큰 마스크와 다르게 어떤 위치의 토큰이 삭제됐는지도 맞춰야함
 - 입력문장에서 불필요/중요치 않은 정보 자동으로 필터링해 처리 가능
⇒ 예측 시간 , 일반화 성능 

3. 문장 순열 (Sentence Permutation)

- 마침표를 기준으로 문장을 나눈 뒤 문장의 순서를 섞는 방법
- 문장의 원래 순서를 맞춰야 함.
- 문장 내 단어들이 어떻게 연결돼 있는지 더 잘 파악하게 돼
입력 문장이 다른 순서로 주어졌을 때도 잘 처리

4. 문서 회전 (Document Rotation)

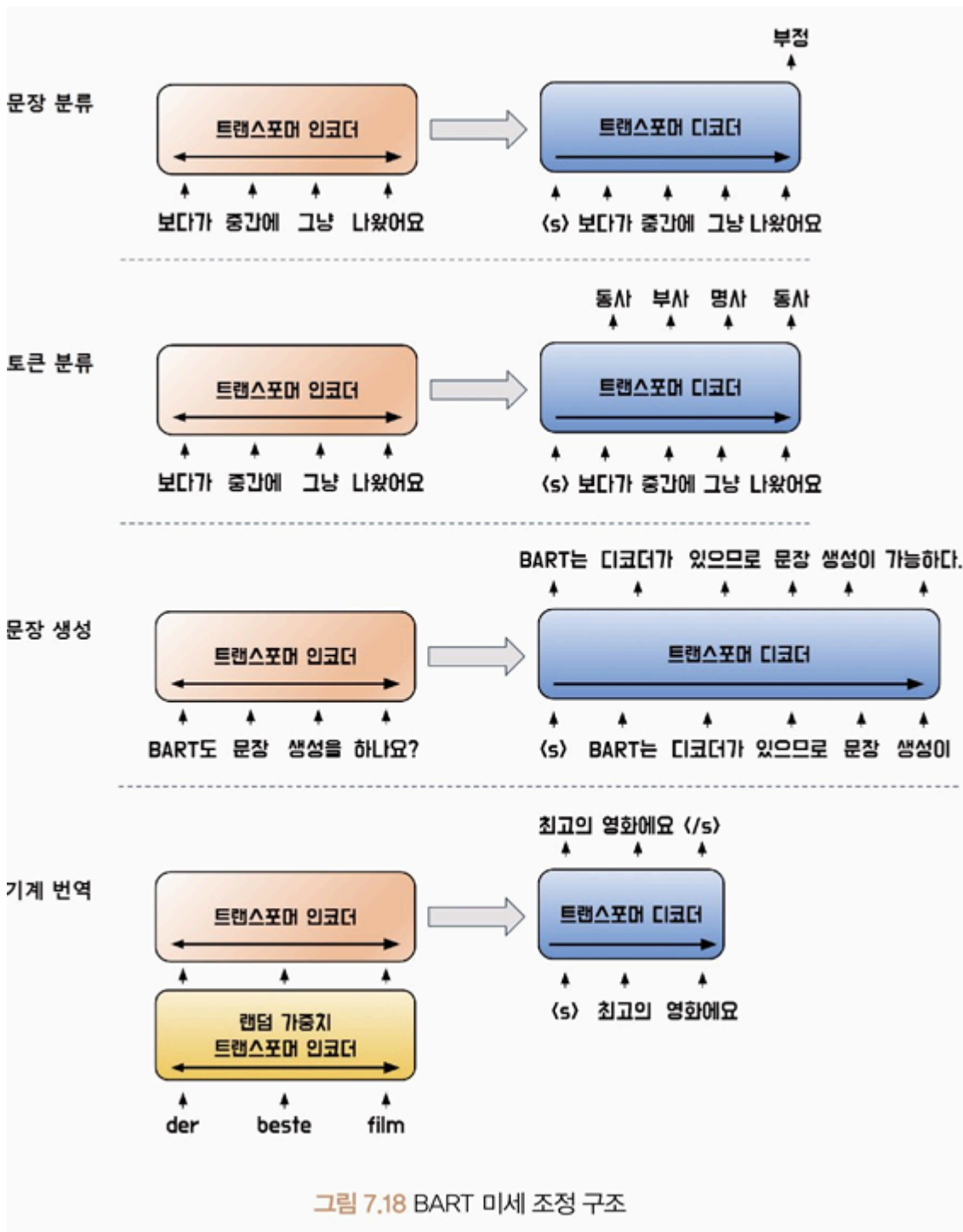
- 임의의 토큰으로 문서가 시작되게 함
- 문장 순열과 다르게 문장의 순서 유지
- 모델은 문서의 원래 시작 토큰을 맞춰야 함
- 모델이 문서의 시작점을 인식할 수 있게 함.

5. 텍스트 채우기 (Text Infilling)

- 몇개의 토큰을 하나의 구간(span)으로 묶고 일부 구간을 마스크 토큰으로 대체
- 묶은 구간의 길이(개수) = 0 ~ 임의의 값 설정 가능
 - 일반적으로 포아송 분포로 구간 나눔, $\lambda = 3$ 으로 설정해 계산
 - 구간의 길이 = 0일 때 해당 위치에 변환된 토큰이 없음
- 모델이 연속된 마스크 토큰 + 마스크 되지 않은 토큰도 구분해야함.
- 모델이 누락된 단어를 예측하게 유도 $\Rightarrow$ 입력 문장 더 잘 이해할 수 있게 됨.

7.4.2 미세 조정 방법

- 인코더 + 디코더 $\Rightarrow$ 다운 스트림 작업에 맞게 입력 문장을 구성해야함
- 인코더 입력 문장 구조 $\neq$ 디코더 입력 문장 구조



- 문장 분류
 - 입력 문장 : 인코더 = 디코더
 - 디코더의 마지막 토큰 은닉 상태를 선형 분류기의 입력값으로 상요
 - BERT의 CLS 토큰과 비슷하지만 BART는 전체 입력과 어텐션 연산이 적용된 은닉 상태 사용
- 토큰 분류

- 입력 문장 : 인코더 = 디코더
- 디코더의 각 시점별 마지막 은닉 상태를 토큰 분류기의 입력 값으로 사용
- 전체 입력과 디코더의 각 시점별 은닉 상태와의 어텐션 연산 수행
- 트랜스포머 디코더에 적합한 작업 (BERT와 비교 - 문장 생성 가능)
- 입력값을 조작해 출력 생성
 - 추상적 질의응답(Abstractive Question Answering)
 - 문장 요약 Summarization
- ⇒ BART의 사전 학습 방식과 유사해 뛰어난 성능을 보임.
- 문장의 의미 파악, 이를 기반으로 새로운 문장 생성
 - e.g. 기계 번역 - BART
 - 사전 학습된 인코더에 기계 번역을 위한 인코더 추가
 - 추가된 인코더 : 기존의 단어 사전 사용 ❌
 - 디코더 : 사전 학습된 가중치, 단어 사전 사용 ◦
 - 학습 단계
 - a. 새로 추가 된 트랜스포머 인코더의 가중치, 위치 임베딩, 첫번째 인코딩 층의 셀프 어텐션 입력 행렬 가중치만 학습
 - b. 모든 신경망의 가중치를 작은 반복으로 학습

 **ROUGE 점수 (N, L, LSUM, W 등)**

정답 문장 유니그램	[대한민국, 은, 16, 강, 에, 진출, 했다]
생성된 문장 유니그램	[대한민국, 은, 8, 강, 에, 진출, 하지, 못, 했다]
ROUGE-1 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{6}{7}$
ROUGE-1 정밀도	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{생성된 문장에 등장한 토큰}} = \frac{6}{9}$
정답 문장 바이그램	[대한민국은, 은 16, 16강, 강에, 에 진출, 진출했다]
생성된 문장 바이그램	[대한민국은, 은 8, 8강, 강에, 에 진출, 진출하지, 하지 못, 못 했다]
ROUGE-2 재현율	$\frac{\text{생성된 문장과 정답 문장에 등장한 토큰}}{\text{정답 문장에 등장한 토큰}} = \frac{3}{6}$
그림 7.19 루지 점수 계산 방법	

1. ROUGE-L

- 생성된 요약문과 정답 요약문 사이 LCS(Longest Common Subsequence, 최장 공통부분 수열)기반의 통계 방식
- LCS : 두 문장 사이 공통적으로 존재하는 가장 긴 부분 문자열을 찾는 문제
 - 문장의 구조적 유사성 고려, 가장 길게 연속되는 N-gram 식별
 - 요약 문장이 입력문장의 의미를 잘 전달하는 지 평가

2. ROUGE-LSUM

- 1.의 변형, 텍스트 내의 개행 문자를 문장 경계로 인식 → 각 문장쌍에 대해 LCS 계산 → union_LCS 계산
- union_LCS : 각 문장쌍의 LCS를 합집합한것, 중복된 부분 제거 후 길이 계산

- 텍스트 생성 작업에서 요약의 정확성과 완전성을 모두 반영할 수 있는 지표로 사용됨

3. ROUGE-W

- 가중치가 적용된 LCS(Weighted LCS-based) 방법
- 연속된 LCS에 가중치를 부여해 계산함.
- 공통 부분 문자열의 길이뿐만 아니라 해당 부분 문자열 내의 단어에 가중치를 부여
- 단어간 유사도를 고려해 요약 문장이 입력 문장의 의미를 더욱 잘 전달하는지 평가하는데 유용

7.5 ELECTRA

Efficiently Learning an Encoder that Classifies Token Replacements Accurately

- MASK 대신 생성자(Generator)와 판별자(Discriminator) 사용해 사전학습 수행

7.5.1 사전 학습 방법

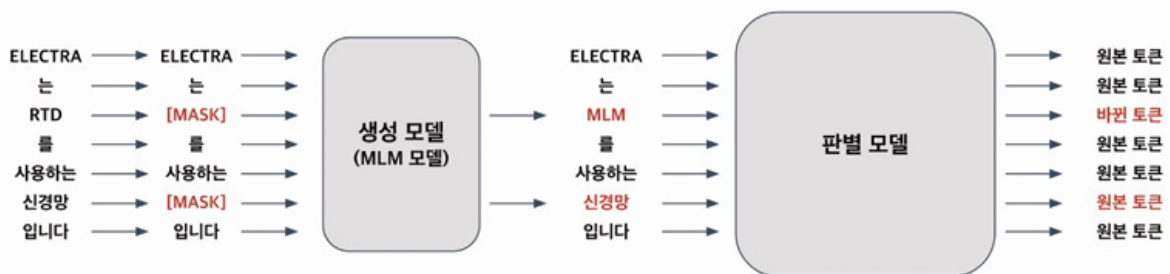


그림 7.20 ELECTRA 모델 학습 방법

✓ 생성자 모델

- 트랜스포머 인코더 구조
- BERT의 마스킹된 언어모델과 동일
- 입력 문장의 15% 마스킹, 원래 어떤 토큰이었는지를 맞히며 학습

✓ 판별자 모델

- 트랜스포머 인코더 구조, BERT와 동일

- 생성자, 판별자가 모두 인코더 구조, 같은 개수의 계층으로 구성될시 가중치 공유 가능
- but, 가중치를 100% 공유하면 생성모델의 성능이 너무 높아져 판별 모델이 학습할 수 없게 됨.
⇒ 생성 모델의 크기를 판별 모델의 1/2-1/4로 설정, 임베딩 계층의 가중치만 공유
- RTD(Replaced Token Detection)
: 각 입력 토큰이 원본 문장의 토큰인지 생성자 모델로 인해 바뀐 토큰인지 맞히며 학습
- 생성자 모델이 복원한 결괏값을 입력받아 각 토큰이 원본 토큰인지 바뀐 토큰인지를 판별
- 사전 학습이 완료되면 생성 모델을 사용하지 않고 오직 판별 모델만 사용해 다운스트림 작업 수행
⇒ BERT와 동일한 방식으로 미세조정

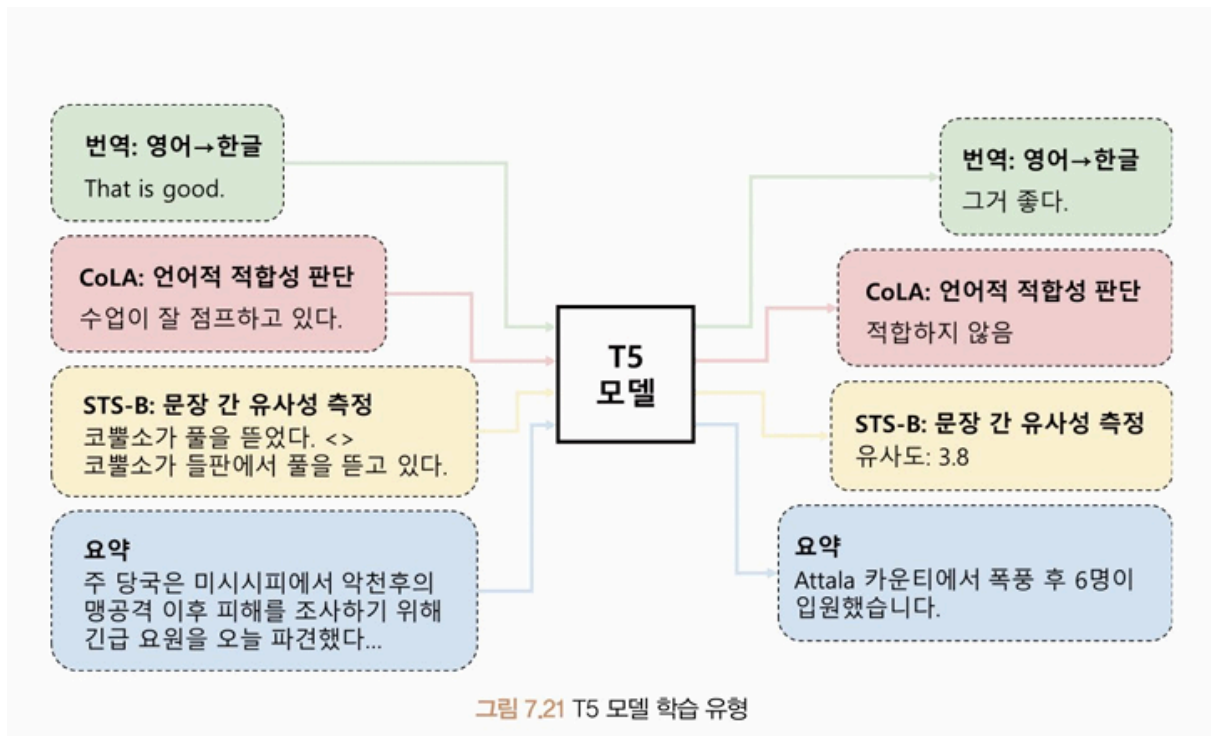
✅ GAN(생성적 적대 신경망)과의 차이점

1. ELECTRA - 생성자 모델이 원래 토큰을 정확히 예측했을 때 이 토큰은 원본 토큰으로 인식함.
2. GAN - 생성 모델과 판별 모델을 적대적으로 학습
⇒ 생성 모델은 판별 모델이 실제 데이터와 구분하기 어렵게 학습
⇒ 판별 모델은 실제 데이터와 생성된 데이터를 더욱 잘 구분하게 학습
3. GAN - 완전한 노이즈 벡터를 입력받아 생성, ELECTRA - 일부 토큰이 마스킹된 텍스트를 입력받음

7.6 T5

Text-to-Text Transfer Transformer

- 인코더 - 디코더 모델 구조
- GLUE, SuperGLUE, CNN/DM 등의 데이터셋에서 SOTA 달성
- 입력, 출력 : 모두 토큰 시퀀스로 처리하는 텍스트-텍스트(Text-to-Text) 구조
 - 입출력의 형태가 자유로움
 - 모델 구조상 유연성과 확장성이 뛰어남



- CoLA 데이터셋 : 문법적으로 허용 가능한 문장과 그렇지 않은 문장 구분 가능
- STS-B 데이터셋 : 문장 간 의미적 유사성 측정 가능
- C4 데이터셋 : 사전학습된 데이터셋
- 사전학습 - 비지도 학습 방식
 - 입력 문장의 일부 구간을 마스킹해 입력 시퀀스 처리
 - 출력 시퀀스 : 실제 마스킹된 토큰과 마스크 토큰의 연결로 구성
 - 센티널 토큰 : 문장마다 유일한 마스크 토큰 의미 ⇒ 센티널 토큰 예측을 목적으로 사전학습됨
- 미세 조정 - 지도 학습 방식
 - 입력과 출력은 텍스트 시퀀스 토큰들로 이루어짐, 인코더-디코더 T5 모델에 입력
 - 문장이 인코딩 되기전 작업토큰을 문장 앞에 추가하면 작업토큰도 같이 학습해 높은 성능.